



To Tag, or Not to Tag: Translating C's Unions to Rust's Tagged Unions

Jaemin Hong

KAIST

Daejeon, South Korea

jaemin.hong@kaist.ac.kr

Sukyoung Ryu

KAIST

Daejeon, South Korea

sryu.cs@kaist.ac.kr

ABSTRACT

Automatic C-to-Rust translation is a promising way to enhance the reliability of legacy system software. However, C2Rust, an industrially developed translator, generates Rust code with unsafe features, undermining the translation's objective. While researchers have proposed techniques to remove unsafe features in C2Rust-generated code, these efforts have targeted only a limited subset of unsafe features. One important unsafe feature remaining unaddressed is a *union*, a type consisting of multiple fields sharing the same memory storage. Programmers often place a union with a *tag* in a struct to record the last-written field, but they can still access wrong fields. In contrast, Rust's *tagged unions* combine tags and unions at the language level, ensuring correct value access. In this work, we propose techniques to replace unions with tagged unions during C-to-Rust translation. We develop a static analysis that facilitates such replacement by identifying tag fields and the corresponding tag values. The analysis involves a must-points-to analysis computing struct field values and a heuristic interpreting these results. To enhance efficiency, we adopt intraprocedural function-wise analysis, allowing selective analysis of functions. Our evaluation on 36 real-world C programs shows that the proposed approach is (1) precise, identifying 74 tag fields with no false positives and only five false negatives, (2) mostly correct, with 17 out of 23 programs passing tests post-transformation, and (3) efficient, capable of analyzing and transforming 141k LOC in 4,910 seconds.

CCS CONCEPTS

• **Software and its engineering** → **Source code generation**; *Automated static analysis*; Maintaining software; Software evolution.

KEYWORDS

Rust, C, Automatic Translation, Union, Tagged Union

ACM Reference Format:

Jaemin Hong and Sukyoung Ryu. 2024. To Tag, or Not to Tag: Translating C's Unions to Rust's Tagged Unions. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, October 27–November 1, 2024, Sacramento, CA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3691620.3694985>



This work is licensed under a Creative Commons Attribution International 4.0 License. ASE '24, October 27–November 1, 2024, Sacramento, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1248-7/24/10

<https://doi.org/10.1145/3691620.3694985>

1 INTRODUCTION

Translating C code to Rust is a promising approach to enhancing the reliability of legacy system software. C Programs often suffer from memory bugs leading to critical security vulnerabilities due to the absence of language-level mechanisms to prevent them [4, 43]. Rust, a recently developed system programming language, ensures memory safety at compile time through type checking [18, 23]. By translating legacy C code to Rust, developers can detect previously unknown bugs and prevent introducing new bugs [16].

Since manual translation is laborious and error-prone, an automatic C-to-Rust translator named C2Rust [46] has been developed in the industry. It converts C code to Rust by leveraging *Unsafe Rust* [37], which allows the use of *unsafe* language features. These features, such as dereferencing raw pointers and calling functions in external code, are equivalent to C's features and enable straightforward syntactic translation. However, as the compiler does not ensure their safety, their use contradicts the goal of translation.

To address this, researchers have proposed techniques to reduce the use of unsafe features in C2Rust-generated code by replacing them with safe counterparts in Rust. LAERTES [7, 8] and CROWN [48] replace raw pointers with references, whose validity is guaranteed by the compiler. Concrat [14] replaces certain external function calls by substituting the C lock API with the Rust lock API. Unfortunately, raw pointers and external functions are not the only sources of unsafety, and previous studies have neglected other unsafe features, limiting applicability to the translation of real-world C code.

Unions are an important source of unsafety in C-to-Rust translation that has not been studied yet. A union is a compound data type consisting of multiple fields sharing the same memory storage, facilitating efficient memory use by allowing values of different types to be stored at the same location [38]. Since memory efficiency is crucial in system software, unions are widely used in C. Notably, Emre et al. [8] show that 18% of unsafe functions (functions using unsafe features) in C2Rust-generated code involve unions.

Reading a union field is an unsafe feature in Rust because unions do not record which field has been written to. If a program reads a field other than the last-written one, the value is *reinterpreted* as another type. While reinterpretation is useful for some uses, like packet parsing, it is dangerous in general. For example, reinterpreting an integer as a pointer can lead to invalid memory access. Thus, many C programs avoid reinterpretation when using unions.

To use unions without reinterpretation, it is essential to decide which field to read. Some programs rely on global context to determine the field, but many use *tags*, i.e., integer values signifying the last-written fields. When using tags, a union and a tag are placed in a single struct, and the program checks the tag before accessing the union's field. However, tags cannot guarantee the absence of

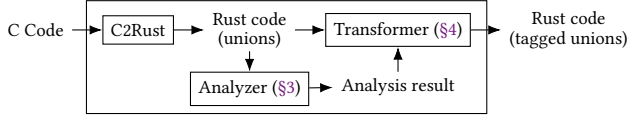


Figure 1: The workflow of the proposed approach

reinterpretation. Programs may read wrong fields after checking tags or set incorrect tag values when writing to fields.

Rust directly supports this pattern of combining tags and unions as a language feature called *tagged unions* (or *enums*) [35]. This allows defining a tagged union as a single type by enumerating tags and the type of a value associated with each tag. By using tagged unions, programmers can avoid mistakenly reinterpreting values. To access a value in a tagged union, programs must use *pattern matching*, which checks the tag and provides access to the associated value. When constructing a tagged union, the compiler ensures that the tag and the value’s type match the type definition. Thus, tagged unions are a safe feature in Rust, making it desirable to replace unions accompanied by tags with tagged unions.

In this work, we propose techniques to translate C’s unions to Rust’s tagged unions. Fig. 1 shows the workflow of the proposed approach. We first translate C code to Rust code that still contains unions using C2Rust. We then transform the C2Rust-generated code by replacing unions with tagged unions. To enable this transformation, we perform static analysis to obtain information related to unions: (1) the *tag field* (the field containing a tag value) for each union and (2) tag values associated with each union field. This static analysis must meet several challenging requirements.

First, the analysis needs to determine the values of struct fields. Programs typically use *switch/if* to access different union fields in different branches, using the tag field in the condition. If a struct field has distinct values when accessing different union fields, it is likely a tag field, and each distinct value is associated with the accessed union field. If it has the same value when accessing different union fields, it is not a tag field. Thus, we can identify tag fields by deciding the value of each struct field in *switch/if* branches.

To achieve this goal, we propose a *must-points-to analysis* capable of tracking *integer equality*. To determine a struct’s field value at each program point based on the branch that the program point belongs to, the struct should be the same as the struct whose field is used in the *switch/if* condition. Since structs are often passed as pointers, deciding whether they are the same requires *must-points-to* relations. Additionally, programs sometimes use a local variable storing a field’s value in a condition. In such cases, determining the field’s value in each branch requires the knowledge that the field and the local variable have the same integer.

The second requirement for the analysis is efficiency. The key idea for achieving efficiency is to selectively analyze functions. To identify tag fields, we need the field values only of the structs containing unions. It means that functions not accessing such structs do not need to be analyzed. Therefore, we adopt intraprocedural function-wise analysis, instead of interprocedural whole-program analysis, allowing only the selected functions to be analyzed.

The third requirement is the ability to identify tag values associated with each union field despite the imprecision of the analysis. Given that imprecision is a fundamental limitation of static analysis [28], it may not be possible to determine the field values at some

program points. To address this, we propose a heuristic to interpret such partial information. The heuristic involves two steps. First, we examine the accessed union fields and the struct field values in *switch/if* branches. This provides reliable information because programs typically access the correct union field after checking the tag. However, due to imprecision, it may fail to identify some tag values. Second, we inspect the last-written union fields and the struct field values at each program point. This can capture information missed by the first step but may be incorrect because of an *intermediate* state, where only the tag or the union field has been set. Therefore, we ignore the field associated with a tag by the second step if it differs from the one associated with the tag by the first step.

Overall, the contributions of this work are as follows:

- We propose static analysis that identifies tag fields and tag values associated with union fields, consisting of *must-points-to* analysis capable of tracking integer equality and a heuristic interpreting the analysis results (§3).
- We propose code transformation replacing unions with tagged unions using the analysis results (§4).
- We implement the proposed approach in a tool named *Urcrat* (*union-removing C-to-Rust automatic translator*) and evaluate it using 36 real-world C programs. Our evaluation shows that the approach is (1) precise, identifying 74 tag fields with no false positives and only five false negatives, (2) mostly correct, with 17 out of 23 programs passing tests after transformation, and (3) efficient, capable of analyzing and transforming 141k LOC in 4,910 seconds (§5).

We also discuss related work (§6) and conclude the paper (§7).

2 BACKGROUND

In this section, we briefly describe the use of unions with tags in C (§2.1), how C2Rust translates such C code to Rust (§2.2), and how tagged unions in Rust can safely represent the same logic (§2.3).

2.1 Unions with Tags

As an example of C code using unions, we use the syntax and evaluation of simple arithmetic expressions defined as follows:

$$e ::= 1 \mid -e \mid e + e \mid e * e$$

An expression is either a constant 1, a negation, an addition, or a multiplication. This syntax is implemented as follows:

```

struct Expr {
  int kind; union { struct Expr *e; struct BExpr b; } v;
};
struct BExpr { struct Expr *l; struct Expr *r; };
  
```

`struct Expr` is the type of an expression, and its field `kind` indicates the kind of expression: 0 for constant 1, 1 for negation, 2 for addition, and 3 for multiplication. The inner union value `v` stores the necessary data for each kind of expression. When `kind` is 1, the operand of negation is stored in `v.e`; when `kind` is 2 or 3, the operands are stored in `v.b.l` and `v.b.r`. Since `kind` signifies which union field has been written to, it is the tag field for the union.

A function evaluating an expression is implemented as follows:

```

int eval(struct Expr *e) {
  switch (e->kind) {
    case 0: return 1;
    case 1: return -eval(e->v.e);
    case 2: return eval(e->v.b.l) + eval(e->v.b.r);
  }
}
  
```

```
case 3: return eval(e->v.b.l) * eval(e->v.b.r);
default: exit(1); }
```

It evaluates the expression by checking the kind field and accessing the appropriate union field accordingly. When kind is 1, it accesses the union field e; when kind is 2 or 3, it accesses b. No field is accessed otherwise.

Programs sometimes use if to check tags, particularly to compare with a specific tag value. An example using if is shown below:

```
if (e->kind == 1) return -eval(e->v.e);
```

While tag fields are beneficial for accessing the correct union fields, they cannot prevent memory bugs. For example, the following code accesses `e->v.b` despite kind being 1:

```
switch (e->kind) {
case 1: return eval(e->v.b.l) + eval(e->v.b.r);
```

Here, `e->v.b.l` accesses the pointer stored in `e->v.e`, but `e->v.b.r` reads an arbitrary value, potentially causing invalid memory access. Furthermore, even when `eval` is correctly implemented, an incorrect tag value can be assigned during the construction of an `Expr`. The following code sets `e.kind` to 2 but writes to `e.v.e`:

```
struct Expr e; e.kind = 2; e.v.e = ...;
```

Passing a pointer to `e` to `eval` results in invalid memory access.

2.2 C2Rust's Translation

C2Rust translates the definition of `struct Expr` to Rust as follows:

```
struct Expr { kind: i32, v: C2RustUnnamed }
union C2RustUnnamed { e: *mut Expr, b: BExpr }
struct BExpr { l: *mut Expr, r: *mut Expr }
```

Since Rust does not support anonymous types, C2Rust gives the name `C2RustUnnamed` to the union. If a file contains multiple anonymous types, they get `C2RustUnnamed_n` where `n` is a unique integer.

The function `eval` is translated as follows:

```
fn eval(e: *mut Expr) -> i32 {
match (*e).kind {
0 => return 1,
1 => return -eval((*e).v.e),
2 => return eval((*e).v.b.l) + eval((*e).v.b.r),
3 => return eval((*e).v.b.l) * eval((*e).v.b.r),
_ => exit(1), }
```

Since Rust provides match statements instead of switch, the function employs match to check the tag. While match is mainly used for pattern matching on tagged unions, it can also handle integers, similar to switch, but without fall-through behavior.

2.3 Tagged Unions

We can implement the same syntax using tagged unions as follows:

```
struct Expr { v: C2RustUnnamed }
enum C2RustUnnamed {
One, Neg(*mut Expr), Add(BExpr), Mul(BExpr) }
struct BExpr { l: *mut Expr, r: *mut Expr }
```

In Rust, the `enum` keyword defines tagged unions. Although the name `C2RustUnnamed` is impractical, we retain it for consistency with the C code. A tagged union's definition lists its *variants*, i.e., values with distinct tags. Each tag is an identifier, not an integer, and the type of a value associated with each tag is specified after the tag. The defined tagged union has four variants with tags `One`, `Neg`, `Add`, and `Mul`. The `Neg` tag is associated with an `Expr` pointer, while `Add` and `Mul` are associated with a `BExpr` value. Since `C2RustUnnamed` now contains the tag, the `kind` field in the struct is no longer necessary.

Now, `eval` can be implemented with pattern matching as follows:

```
1 fn eval(e: *mut Expr) -> i32 {
2   match (*e).v {
3     C2RustUnnamed::One => return 1,
4     C2RustUnnamed::Neg(e) => return -eval(e),
5     C2RustUnnamed::Add(b) => return eval(b.l)+eval(b.r),
6     C2RustUnnamed::Mul(b) => return eval(b.l)*eval(b.r),
7   }
```

Each pattern matching branch specifies a tag and binds the associated value to an identifier. For instance, in line 4, the associated value is bound to `e` and then passed to `eval`. Since the compiler ensures all variants are covered, the `_` (default) branch is unnecessary.

The code pattern using if to check tags is also supported through `if-let` [34], which is another form of pattern matching. The following code performs computation only when the tag is `Neg`:

```
if let C2RustUnnamed::Neg(e) = (*e).v {
return -eval(e); }
```

Pattern matching enables the compiler to detect programmers' mistakes that can cause memory bugs. Consider the following Rust code where the value associated with `Neg` is treated as a `BExpr` type:

```
match (*e).v {
C2RustUnnamed::Neg(b) => return eval(b.l) + eval(b.r),
```

The type of `b` is `*mut Expr`, as specified in the type definition. Since `*mut Expr` lacks the fields `l` and `r`, the compiler raises an error.

Additionally, the compiler verifies the correct construction of tagged union values. Consider the following code, which incorrectly initializes a tagged union with the tag `Add` and an `Expr` pointer:

```
let e1: Expr = ...;
let e2 = Expr { v: C2RustUnnamed::Add(&mut e1) };
```

Since the type definition requires a `BExpr` value for `Add`, the code does not pass type checking.

3 STATIC ANALYSIS

In this section, we present static analysis designed to facilitate the transformation of unions with tags into tagged unions. The objectives of this analysis are to (1) identify a tag field for a union, if one exists, and (2) determine the tag values associated with each union field. The proposed static analysis consists of four steps:

- (1) Identification of *candidate structs*, those containing unions and their potential tag fields (§3.1).
- (2) Whole-program may-points-to analysis (§3.2).
- (3) Intraprocedural must-points-to analysis for selected functions (§3.3).
- (4) Interpretation of the analysis results using a heuristic (§3.4).

As we analyze Rust code generated by C2Rust, rather than the original C code, code examples in this section are written in Rust.

3.1 Candidate Identification

The first step of the analysis is to identify structs that likely contain unions and their tag fields. We also determine which functions to analyze based on the identified structs. If no such structs are found, the analysis terminates at this step, concluding that the program does not contain unions to be transformed into tagged unions.

To concretely define candidate structs, we first define *tag-eligible fields*, those that can potentially serve as tag fields. A field of a struct is considered tag-eligible if (1) it has an integer type, (2) when it appears on the left side of an assignment, the operator is `=`, excluding others such as `+=`, and (3) it is never referenced by a pointer. For the first condition, integer types include `bool`, `i8`, `u8`,

i16, u16, i32, u32, i64, and u64, which C2Rust translates from `_Bool`, signed/unsigned char, short, int, and long. The second and third conditions arise from limitations in the expressibility of tagged unions. Tags of tagged unions are not integers and thus cannot undergo integer operations. Moreover, since tags do not exist as fields, they cannot be referenced. Consequently, fields that exhibit such behavior cannot be transformed into tagged unions.

We also define *candidate unions*, which can potentially be accompanied by tags. A union is a candidate if (1) it is a field of a struct that contains at least one tag-eligible field, and (2) its name begins with `C2RustUnnamed`. The second condition indicates that the union is anonymous in the C code. If a union has a name, it can be used independently of the struct, not being expressible as tagged unions.

We finally define candidate structs. A candidate struct is a struct containing at least one candidate union.

We now describe how to determine which functions to analyze. To identify tag fields for unions and values associated with union fields, we need to ascertain the possible values of candidate struct fields. If a function neither reads from nor writes to a field, intraprocedural analysis of the function provides no information about that field's value. Therefore, we analyze only the functions that access fields of candidate structs. Such functions can be easily identified through syntactic examination.

3.2 May-Points-To Analysis

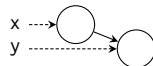
The second step of the analysis is to conduct a whole-program may-points-to analysis. This step is essential because may-points-to relations are required to ensure the soundness of the subsequent must-points-to analysis. The utilization of may-points-to relations in the must-points-to analysis is detailed in §3.3.

We employ a field-sensitive Andersen-style analysis [26] for the may-points-to analysis. This analysis is flow-insensitive and has a time complexity of $O(n^3)$. Although other may-points-to analyses exist, such as field-insensitive Andersen-style [2] and Steensgaard-style analyses [40, 41], they are too imprecise, leading to unacceptably imprecise must-points-to analysis results.

3.3 Must-Points-To Analysis

The third step is to perform an intraprocedural must-points-to analysis for each selected function. Our overall algorithm is akin to typical must-points-to analyses [19]. The execution state at each program point is represented as a graph, with nodes denoting memory locations and edges expressing points-to relations. The analysis iteratively updates the state at each program point until reaching a fixed point. Each state is derived by joining the state from the previous iteration with the state resulting from applying the current instruction's effect to the previous program point's state.

We first describe how we visualize graphs throughout this section. Since nodes denote memory locations, some nodes correspond to the stack locations used by local variables. For clarity in visualization, we draw a dashed arrow from the name of a local variable to the node representing its memory location. Note that a variable name is not a node and thus this arrow is not an edge. For example, `x = &y` constructs the following graph:

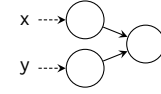


The graph has two nodes and one edge and indicates that the pointer at the memory location of `x` points to the memory location of `y`.

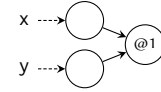
Since our goal is not only to compute must-points-to relations but also to track fields' integer values, we optionally label each node with `@N`, where `N` is a set of integers. The label `@N` encodes integer value information in pointer graphs. If a node is unlabeled, the value at the location is a usual C value (such as an integer, pointer, struct, union, or array). If a node is labeled `@N`, the value at the location is an imaginary value at addresses `N`, i.e., the possible addresses of the location are `N`. Consequently, if a node `v` has an outgoing edge to a node labeled `@N`, then the possible values at `v`'s location are `N`. For instance, `x = 1` constructs the following graph:



Using the `@N` label is beneficial since it enables efficient propagation of integer values to memory locations known to hold the same value. Consider the following graph, constructed by `x = y`:



If we obtain the fact that `x` equals 1, we update the graph as follows:

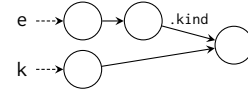


Then, we automatically discover that `y` also equals 1. As this example demonstrates, the `@N` label facilitates the update of the value at multiple memory locations by labeling only a single node.

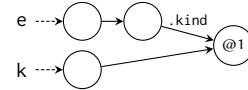
We now discuss how to analyze code involving unions. Consider the following example, where `Expr` is defined as in §2.2:

```
1 fn eval(e: *mut Expr) -> i32 {
2   let k = (*e).kind;
3   match k {
4     1 => return -eval((*e).v.e);
```

After line 2, we have the following graph:



Here, the edge labeled `.kind` indicates the presence of a struct/union at the location of the node where the edge originates, with the pointer stored in the `kind` field referring to the pointed node's location. Since line 3 uses `k` as the condition for `match`, we determine that `k` equals 1 in line 4. Consequently, the graph at the beginning of line 4 is as follows:



From this graph, we conclude that the value of `kind` is 1 when the union field `e` is accessed in line 4.

More complex scenarios involve joining two graphs. Consider the following code example:

```
1 match (*e).kind {
2   2 => ...,
3   3 => ...,
4   _ => return, }
5 let lv = eval((*e).v.b.1);
```

In lines 2 and 3, we have the following graphs, respectively:

Algorithm 1: Graph joining

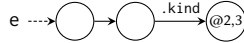
Input : g_1, g_2
Output : g

```

1  $g.nodes := \emptyset; g.edges := \emptyset; \text{worklist} := \emptyset;$ 
2 for  $x \leftarrow \text{function's local variables};$ 
3   if  $g_1$  has a node  $v_1$  corresponding to  $x$ :
4     if  $g_2$  has a node  $v_2$  corresponding to  $x$ :
5        $g.nodes.insert((v_1, v_2));$ 
6        $\text{worklist.insert}((v_1, v_2));$ 
7 while  $\text{worklist} \neq \emptyset$ :
8    $(v_1, v_2) := \text{worklist.pop}();$ 
9   for  $(v_1, v'_1, f) \leftarrow v_1$ 's outgoing edges in  $g_1$ :
10    if  $g_2$  has an edge  $(v_2, v'_2, f)$ :
11       $g.edges.insert((v_1, v'_1, f), (v_2, v'_2, f));$ 
12      if  $(v'_1, v'_2) \notin g.nodes$ :
13         $g.nodes.insert((v'_1, v'_2));$ 
14         $\text{worklist.insert}((v'_1, v'_2));$ 
15  if  $v_1$  has a label  $@N_1$  in  $g_1$ :
16    if  $v_2$  has a label  $@N_2$  in  $g_2$ :
17      set  $(v_1, v_2)$ 's label to  $@(N_1 \cup N_2)$  in  $g$ ;
```



Since line 5 is reachable from both lines 2 and 3, we need to join the graphs. When joining graphs, the integer sets in the labels are unioned, resulting in the following graph:



From the graph, we conclude that the value of `kind` is 2 or 3 when the union field `b` is accessed in line 5.

The join of graphs is carefully defined to maintain the soundness of the analysis. Consider the following code, where control flow splits based on the value of `kind` and then merges:

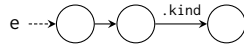
```

1 if  $(*e).kind == 1$  { ... }
2 else { ... }
3 ...
```

The states in lines 1 and 2 are as follows, respectively:



In line 1, $(*e).kind$ is known to be 1. However, in line 2, its value is unknown. When entering line 3, these graphs are joined as follows:



Since the node from line 2 is unlabeled, the joined graph's node is also unlabeled. This indicates that we do not know the value of $(*e).kind$, which is correct. This example shows that no-label signifies no-information regarding the address of the node's location, i.e., it represents $@\mathbb{Z}$ (all integers), not $@\emptyset$ (empty set).

Alg. 1 illustrates the algorithm for graph joining. Edges are intersected to retain must-point-to relations, while integer sets in labels are unioned. In the pseudo-code, each node in the input graphs g_1 and g_2 is denoted by a unique identifier v , and each node in the output graph g is represented by a pair of identifiers (v_1, v_2) . Each edge is represented by (v, v', f) , denoting an edge labeled f from node v to node v' . Unlabeled edges are treated as edges with the empty label ϵ . Initially, g and the worklist, which stores g 's nodes to be visited, are both empty (line 1). Then, we add nodes corresponding to local variables to g (lines 2–6). Finally, we visit

each added node until the worklist is empty (lines 7–8). During the visit, edges with the same label in g_1 and g_2 are added to g (lines 9–14), and node labels are unioned if they exist (lines 15–17).

We now discuss the utilization of may-points-to relations during the must-points-to analysis. Consider the following code:

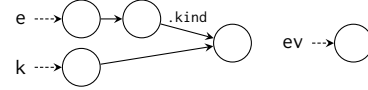
```

1 let  $e = \text{if } \dots \{ \&\text{mut } ev \} \text{ else } \{ \dots \};$ 
2 let  $k = (*e).kind;$ 
3  $ev = \dots;$ 
4 match  $k$  {
5   1 => ... ,
```

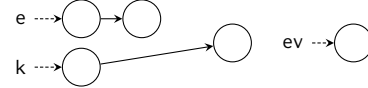
Since e may not point to ev , the state after line 1 is as follows:



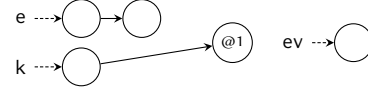
This graph indicates that e points to some location, not necessarily the same as ev 's location. Line 2 updates the graph as follows:



Line 3 mutates the value of ev , possibly changing the value of $(*e).kind$. Thus, $(*e).kind$ is not necessarily equal to k after line 3, necessitating an appropriate update to the graph. As the graph itself does not reveal any relations between e and ev , we rely on the precomputed may-points-to relations, which indicate that e may point to ev . Consequently, we remove all outgoing edges, including the `kind` edge, from $*e$'s node, resulting in the following graph:



In line 5, we successfully avoid the incorrect conclusion that $(*e).kind$ equals 1, as shown in the following graph:



Like this, the analysis removes the appropriate edges from the graph at each (indirect) assignment and function call according to the may-points-to relations. The effect of a function call is equivalent to the cumulative effects of all assignments reachable by the call.

3.4 Analysis Result Interpretation

The last step of the analysis is to interpret the results of the must-points-to analysis using a heuristic, as demonstrated in Alg. 2. The entry point is `IdentifyTags`, which takes three arguments: a candidate struct s , a candidate union u in s , and a tag-eligible field f_s in s . Its goal is to determine whether f_s serves as a tag field for u and, if it does, to identify the tag values associated with each field of u .

As the first step of the heuristic, we invoke `CollectFromAccesses` (line 1) to identify the associated tag values by examining the value of f_s when a union field is accessed after f_s has been checked by `match/if`. This subroutine returns `field_tags`, a map from union fields to their tag values, and `all_tags`, a set containing all tag values in `field_tags`. Initially, both are empty (line 18). We then iterate over every program point in the analyzed functions and check if any field of u is accessed (lines 19–20). If accessed, we determine the possible values of f_s from the `@N` label of the graph computed by the must-points-to analysis (line 21). Here, we treat no-label as the

Algorithm 2: Identifying tag values associated with fields

```

1 def IdentifyTags(struct s, union u, field fs):
2   res := CollectFromAccesses(s, u, fs);
3   if res = None:
4     return None;
5   (field_tags, access_tags) := res;
6   field_tags' := CollectFromStructs(s, u, fs);
7   struct_tags := 0;
8   for fu ← fields of u:
9     tags := field_tags'[fu] \ access_tags;
10    if struct_tags ∩ tags ≠ 0:
11      return None;
12    field_tags[fu] := field_tags[fu] ∪ tags;
13    struct_tags := struct_tags ∪ tags;
14  all_tags := CollectAllTags(s, u, fs);
15  rem_tags := all_tags \ (access_tags ∪ struct_tags);
16  return (field_tags, rem_tags);
17 def CollectFromAccesses(struct s, union u, field fs):
18  field_tags := Map(); all_tags := 0;
19  for l ← analyzed program points:
20    if a field fu of u is accessed at l:
21      N := possible values of fs at l;
22      if N is from if or match:
23        tags := N \ field_tags[fu];
24        if all_tags ∩ tags ≠ 0:
25          return None;
26        field_tags[fu] := field_tags[fu] ∪ tags;
27      all_tags := all_tags ∪ tags;
28  return (field_tags, all_tags);
29 def CollectFromStructs(struct s, union u, field fs):
30  field_tags := Map();
31  for l ← analyzed program points:
32    if l is end of a basic block:
33      for v ← struct s reachable at l:
34        if union u in v has a field fu:
35          N := possible values of fs of v;
36          field_tags[fu] := field_tags[fu] ∪ N;
37  return field_tags;
38 def CollectAllTags(struct s, union u, field fs):
39  all_tags := 0;
40  for l ← analyzed program points:
41    for v ← struct s reachable at l:
42      N := possible values of fs of v;
43      all_tags := all_tags ∪ N;
44  return all_tags;

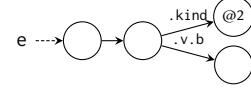
```

empty set. The possible values should originate from match/if on f_s , and not from an assignment to f_s (line 22). If any of these values are already associated with other fields, we immediately return None, indicating that f_s is not a tag field (lines 23–25). Otherwise, we add the values to field_tags and all_tags (lines 26–27).

If CollectFromAccesses succeeds, IdentifyTags proceeds to the next step by calling CollectFromStructs (line 6). This complements the previous step by discovering tag-field associations that may have been missed due to the analysis's imprecision. This subroutine considers the field values and the last-written union fields at the end of each basic block. For instance, consider the following code:

```
(*e).kind = 2; (*e).v.b = ...; return e;
```

We have the following graph at return:



From this, we can conclude that kind equals 2 when b is the last-written union field, likely associating 2 with b.

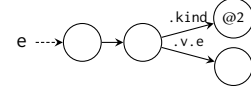
We inspect only the states at the end of basic blocks because the states of other program points are prone to provide incorrect information from *intermediate* states. Consider the following example:

```

1 (*e).kind = 1; (*e).v.e = ...; ...
2 (*e).kind = 2;
3 (*e).v.b = ...; return e;

```

Initially, e is used as a negation expression by setting kind to 1, but it becomes an addition expression by setting kind to 2 in the end. If we examine the state after line 2, we will get an incorrect association between 2 and e from the following graph:



To avoid this issue, we examine only the end of basic blocks, where both the tag value and the union field are likely to be correctly set.

However, this approach cannot completely prevent reading intermediate states. For example, a function call can occur between the tag set and the union field set. Therefore, we prioritize the data from CollectFromAccesses, which are less likely to be affected by intermediate states, over CollectFromStructs. If CollectFromAccesses associates a certain tag value with a specific union field, that tag value in CollectFromStructs' results is ignored.

To implement this approach, CollectFromStructs iterates over the end of every basic block in each analyzed function, identifying every value of type s reachable from local variables at that point (lines 31–33). If the struct's union has a written field, that field is associated with the values of f_s (lines 34–36). Then, IdentifyTags combines results from both subroutines, prioritizing data from CollectFromAccesses (lines 8–13). It removes tags already associated with union fields by CollectFromAccesses from the results of CollectFromStructs (line 9) and returns None if a tag value remains associated with two fields even after this removal (lines 10–11).

Finally, we search for tag values not associated with any union fields. To achieve this, we call CollectAllTags (line 14), which collects all possible tag values by examining the state at every program point (lines 40–43). By removing the tags associated with union fields, we isolate the tags not associated with union fields (line 15).

To determine the tag field for each union, we run IdentifyTags on all tag-eligible fields. The field for which IdentifyTags returns a value other than None is identified as the tag field. If multiple tag fields are found, we select the one with the highest number of distinct tag values.

4 CODE TRANSFORMATION

In this section, we present code transformation that replaces unions with tagged unions in C2Rust-generated code, using the results of the static analysis. We demonstrate the transformation of the Expr type defined in §2.2 as an example. We assume that the tag field and the tag values associated with each union field are correctly identified by the static analysis.

The type definitions are transformed as follows:

```
struct Expr { v: C2RustUnnamed }
enum C2RustUnnamed {
    Empty0, e1(*mut Expr), b2(BExpr), b3(BExpr) }
struct BExpr { l: *mut Expr, r: *mut Expr }
```

This result is the same as the hand-written code in §2.3, except for the variant names. We generate variant names by concatenating the union field name with the tag value. For tags not associated with any fields, we prepend `Empty` to the tag value. To improve variant names, one option is to utilize global variables' names. When using unions with tag fields, C programmers often define an *enum*, i.e., a group of constant integers with associated names, to use them as tag values. These names typically reflect the programmers' understanding of the tag values' meaning, e.g., `EXPR_ONE`. Since `C2Rust` translates each enum definition into multiple constant global variable definitions while preserving the names, we can use these variable names instead of the union field names. Although the names generated with this strategy may still be unsatisfactory, developers can easily rename them to more meaningful names using IDEs.

We now discuss the transformation of code using unions. We propose two approaches: naïve transformation, which can be applied to any code but does not adhere to Rust idioms (§4.1), and idiomatic transformation, which follows Rust idioms but is applicable only to specific code patterns (§4.2). We use both methods within a single codebase, prioritizing idiomatic transformation wherever possible and resorting to naïve transformation when necessary.

4.1 Naïve Transformation

Naïve transformation involves defining helper methods for the transformed structs and unions. These methods are categorized into two groups: reading and writing. We first focus on reading. Below are the read-related methods for `Expr` and `C2RustUnnamed`:

```
1 impl Expr {
2   fn kind(self) -> i32 {
3     match self.v {
4       C2RustUnnamed::Empty0 => 0,
5       C2RustUnnamed::e1(_) => 1, ... }
6   }
7   fn get_e(self) -> *mut Expr {
8     if let C2RustUnnamed::e1(v) = self { v }
9     else { panic!() }}
```

The `kind` method (lines 2–5) of `Expr` replaces the `kind` field in the original code. This method returns the appropriate tag value by applying pattern matching to the tagged union value. Code that reads a tag field is replaced with a call to the tag-returning method.

The `get_e` method (lines 7–9) of `C2RustUnnamed` replaces the union field `e`. This method returns a value by applying pattern matching to the tagged union value. If the current variant does not contain such a value, the method triggers a panic. This allows the dynamic detection of potential bugs by identifying read from a field other than the last-written one, rather than silently reinterpreting the value. Although not shown in the example, a method `get_f` is defined similarly for each union field `f`. We replace code reading a union field with a call to the corresponding getter method.

Using these methods, we transform code reading tags and union fields as follows, where the former represents the code before transformation and the latter represents the code after transformation:

```
match (*e).kind { 1 => eval((*e).v.e), // before
match (*e).kind() { 1 => eval((*e).v.get_e()), // after
```

Although this transformation preserves the semantics, the resulting code is not idiomatic. It applies pattern matching twice to the tagged union value—once to get the tag value and once to get the union field value—instead of applying pattern matching only once to directly access the associated value of each variant.

We now describe the write-related methods, defined as follows:

```
1 impl Expr {
2   fn set_kind(&mut self, v: i32) {
3     match v {
4       0 => { self.v = C2RustUnnamed::Empty0 }
5       1 => {
6         let v =
7           if let C2RustUnnamed::e1(v) = self.v { v }
8           else { std::ptr::null_mut() };
9         self.v = C2RustUnnamed::e1(v) } ... }
10  }
11  fn deref_e_mut(&mut self) -> *mut *mut Expr {
12    if let C2RustUnnamed::e1(_) = self { }
13    else { *self = Self::e1(std::ptr::null_mut()); }
14    if let C2RustUnnamed::e1(v) = self { v }
15    else { panic!() }}
```

The `set_kind` method (lines 2–9) of `Expr` replaces assignments to `kind`. It takes a tag value as an argument and updates the tagged union to the appropriate variant. If the variant has an associated value, we check if this value already exists and reuse it if it does (line 7). If the value does not exist, we create an arbitrary value, which in this case is the null pointer (line 8).

The `deref_e_mut` method (lines 11–15) of `C2RustUnnamed` provides a pointer to the inner value, which we use to replace code that mutates `e` or takes its address. First, we check whether the current variant is appropriate (line 12), and, if not, update the value to the correct variant (line 13). Then, we return a pointer to the value (line 14), while the `panic!()` on line 15 is never reached. We also define a method `deref_f_mut` for each union field `f` in a similar manner.

We transform code that updates tags and union fields as follows:

```
(*e).kind = 1; (*e).v.e = ...; // before
(*e).set_kind(1); (*e).v.deref_e_mut() = ...; // after
```

In the transformed code, `set_kind` changes the variant to `e1`. Then, `deref_e_mut` returns a pointer to the arbitrary value set by `set_kind`, and the indirect assignment to the pointer updates this value.

Note that `deref_e_mut` does not trigger a panic even when the current variant differs from the expected one, unlike `get_e`. This behavior ensures the correct transformation of code that first writes to a union field and then sets the tag. For example, the following transformation preserves the semantics:

```
(*e).v.e = ...; (*e).kind = 1; // before
*(*e).v.deref_e_mut() = ...; (*e).set_kind(1); // after
```

After the transformation, `deref_e_mut` changes the variant to `e1`, and the indirect assignment to the pointer sets the associated value. Then, `set_kind` retains both the variant and associated value. Although this approach preserves the semantics, it is not idiomatic in Rust, as we can create a tagged union value within a single expression instead of setting the tag and the union field separately.

4.2 Idiomatic Transformation

Idiomatic transformation uses pattern matching on tagged union values and constructs a tagged union value with a single expression, avoiding helper methods. Below is the transformation of `match`:

```
match (*e).kind { 1 => eval((*e).v.e), // before
```

```
// after
match (*e).v { C2RustUnnamed::e1(ref x) => eval(*x),
```

The match condition is the tagged union value itself, rather than the tag value obtained by `get_kind`. In addition, each branch directly matches the variant, instead of comparing the tag value to an integer. The `ref` keyword before the identifier `x` binds a pointer to the associated value, not the value itself, to `x`, allowing the branch to read and modify the value. This eliminates the need to call methods such as `get_e` and `deref_e_mut`, as `x` directly accesses the value.

We also transform `if` that checks tag values to use pattern matching with `if-let`, as illustrated in the following example:

```
if (*e).kind == 1 { eval((*e).v.e) } // before
```

```
// after
if let C2RustUnnamed::e1(ref x) = (*e).v { eval(*x) }
```

We handle the disjunction of multiple equality comparisons using `|`, the `or` operator in pattern matching. An example is shown below:

```
if (*e).kind == 2 || (*e).kind == 3 { eval((*e).v.b.l) } // before
```

```
// after
if let C2RustUnnamed::b2(ref x) |
  C2RustUnnamed::b3(ref x) = (*e).v { eval((*x).l) }
```

However, the idiomatic approach is not applicable to other kinds of conditions, such as conjunctions and inequalities, because Rust's patterns currently cannot represent these conditions. A conjunction specifies that the tag is a particular integer and also that some boolean formula is satisfied. When using `match` for pattern matching, Rust offers *match guards* [36] to express such logic. However, when using `if-let`, this logic is supported through *let chains* [9], which is currently an unstable feature and requires a special flag to enable. For this reason, we chose not to transform conjunctions into pattern matching in this work, leaving it for future work once this feature is stabilized. On the other hand, an inequality specifies that the tag is not a particular integer. However, the purpose of pattern matching is to check whether a value conforms to a certain pattern, not that it does not, and thus it cannot replace inequalities. Thus, the following code is transformed using the naïve approach:

```
if (*e).kind == 1 && ... { eval((*e).v.e) }
if (*e).kind != 1 { ... } else { eval((*e).v.e) }
```

The idiomatic transformation is also not applicable when union fields are accessed without using `match` or `if` to check the tag field. C programmers sometimes assume that a tag field has a specific value at a certain point based on their understanding of the program's behavior and directly access the union field without checking the tag. Since tag check is not performed, we cannot transform such code into pattern matching and must resort to the naïve transformation.

The idiomatic transformation consolidates multiple assignment expressions within a single code block into a single assignment expression that constructs a tagged union value if the assignments set the tag and union fields. Below illustrates this transformation:

```
{ ... (*e).kind = 1; (*e).v.e = ...; ... } // before
{ ... (*e).v = C2RustUnnamed::e1(...); ... } // after
```

When multiple assignments are distributed across different code blocks, they are individually transformed using the naïve approach.

5 EVALUATION

In this section, we evaluate our approach with 36 real-world C programs. We first describe our implementation of Urcrat, which realizes the proposed approach (§5.1), and the process of collecting

Table 1: Benchmark programs

Name	C LOC	Rust LOC	#Unions	#Candidates	#Identified
bc-1.07.1	10810	16982	4	1	1
binn-3.0**	5686	4298	1	1	0
brotili-1.0.9**	13173	127691	6	4	0
cflow-1.7	20601	26375	5	4	3
compton*	8748	14084	2	2	2
cpio-2.14	35934	80929	10	4	3
diffutils-3.10	59377	95835	7	5	4
enscript-1.6.6	34868	78749	9	5	3
findutils-4.9.0	80015	139858	13	6	3
gawk-5.2.2	58111	140566	17	10	3
glpk-5.0	71805	145738	18	14	3
gprolog-1.5.0	52193	74381	5	2	0
grep-3.11	64084	84902	11	9	6
gsl-2.7.1	227199	422854	14	14	0
gzip-1.12	20875	21605	4	2	1
hiredis*	7305	14042	1	1	1
make-4.4.1	28911	36336	1	1	1
minilisp*	722	2149	1	1	1
mtools-4.0.43	18266	37021	2	1	0
nano-7.2	42999	74994	6	4	3
nettle-3.9	61835	82742	5	2	1
patch-2.7.6	28215	103839	3	1	1
php-rdkafka*	3771	28864	1	1	1
pocketlang*	14267	41439	4	3	3
pth-2.0.7	7590	12950	1	1	1
raygui*	1588	17218	1	1	1
rcs-5.10.1	28286	36267	1	1	1
screen-4.9.0	39335	72199	1	1	0
sed-4.9	48190	68465	8	7	4
shairport*	4995	10118	2	1	1
tar-1.34	66172	134972	16	12	9
tinyproxy*	5667	12825	5	2	2
twemproxy*	22738	74593	8	7	5
uucp-1.07	51123	77872	3	3	0
webdis*	14369	29474	2	2	2
wget-1.21.4	81188	192742	6	5	4
Total			204	141	74

*: from GitHub, **: from CROWN

the benchmark programs (§5.2). The implementation and benchmark programs are publicly available [15]. We then assess our approach by addressing the following research questions:

- RQ1. Precision and recall: Does it identify tag fields without false positives or false negatives? (§5.3)
- RQ2. Correctness: Does it transform code while preserving its semantics? (§5.4)
- RQ3. Efficiency: Does it efficiently analyze and transform programs? (§5.5)
- RQ4. Code characteristics: How much does the code change due to the transformation, and how frequently are the helper methods called? (§5.6)
- RQ5. Impact on performance: What is the effect of replacing unions with tagged unions on program performance? (§5.7)

Our experiments were conducted on an Ubuntu machine with Intel Core i7-6700K (4 cores, 8 threads, 4GHz) and 32GB DRAM. Finally, we discuss potential threats to validity (§5.8).

5.1 Implementation

We built Urcrat on top of the Rust compiler [31]. Urcrat analyzes Rust code after it has been lowered to Rust's mid-level intermediate representation (MIR) [33], which expresses functions as control flow graphs with basic blocks. For code transformation, Urcrat utilizes Rust's high-level intermediate representation (HIR) [32], akin to abstract syntax trees but with syntactic sugar removed and symbols resolved. We employed C2Rust v0.18.0 with minor modifications.

5.2 Benchmark Program Collection

We collected benchmark programs from three sources: (1) CROWN [48], (2) GNU packages [12], and (3) GitHub. Only 2 out of 20 programs

used by CROWN have candidate unions, so we expanded the benchmarks with additional sources. We chose GNU packages for their representative C projects and GitHub for its diverse code patterns. We avoided large codebases because C2Rust often produces Rust code with type errors, primarily due to missing type casts, which require significant manual effort to correct. From GNU, we gathered C packages with less than 250k LOC and individual Wikipedia entries, indicating they are well-known. From GitHub, we gathered C projects with less than 1 MB of code and over 1,000 stars. In both collections, we retained programs (1) compiled successfully on Ubuntu, (2) transpiled successfully by C2Rust, and (3) containing candidate unions. This resulted in 24 programs from GNU and 10 from GitHub, giving us a total of 36 benchmark programs. Of these, 20 required manual fixes after C2Rust’s translation, with an average of 26.6 lines modified. Columns 2–5 of Table 1 present the C LOC, Rust LOC, number of unions, and number of candidate unions in each program, respectively. In the benchmarks, 21% of unsafe functions involve unions, and 11% involve tagged unions.

5.3 RQ1: Precision and recall

We evaluate the precision and recall of the proposed approach. We first identified the tag field of each candidate union through static analysis. Column 6 of Table 1 shows the number of candidate unions for which a tag field is identified in each benchmark program. Out of 141 candidates across 36 programs, Urcrat identifies tag fields for 74 candidates in 29 programs. We then manually inspected each candidate union to determine the presence of a tag field, checking for false positives (a tag field identified for a union that does not have one) and false negatives (no tag field identified for a union that has one). For practical use of our tool, false positives are problematic because they change the program’s semantics. Conversely, false negatives are less problematic as they only prevent the replacement of unions with tagged unions, and the semantics remains correct.

Our manual inspection shows that the static analysis is precise, revealing no false positives and only five false negatives. This results in a precision of 100% and a recall of 93.7% ($= 74/79$). Specifically, two false negatives occur in *glpk-5.0*, while the others are in *gawk-5.2.2*, *screen-4.9.0*, and *uucp-1.07*, respectively. Three of these (from *glpk-5.0*, *gawk-5.2.2*, and *screen-4.9.0*) are due to intermediate states involving tag values that are not associated with any union fields by `CollectFromAccesses`. We now discuss the reasons for the remaining two false negatives:

glpk-5.0. The false negative arises from `goto` in the C code:

```
switch (tab->type) { case 112: goto input_table;
                  case 119: goto output_table; default: abort(); }
input_table: ... return; output_table: ...
```

In this code, `type` is the tag field. Since Rust does not support `goto`, C2Rust translates the code as follows:

```
match (*tab).type {
  112 => current_block = 1, 119 => { ... } _ => abort(),
  if current_block == 1 { ... }
```

The resulting code has a variable, `current_block`, which mimics `goto`’s effect. As the analysis is path-insensitive, it concludes that both 112 and 119 are possible values in the true branch of `if`, hindering the identification of the tag field. To address this issue, we need to either analyze the original C code or modify the C code to avoid `goto`. We chose the latter and revised the code as follows:

```
switch (tab->type) { case 112: ... break;
                  case 119: ... break; default: abort(); }
```

Urcrat can identify the tag field from the modified version.

uucp-1.07. This false negative is due to a bug in the C code:

```
if (qport->uuconf_ttype == 5) {
  if (qport->uuconf_u.uuconf_stli.zdevice != NULL)
    fprintf(e, "%s", qport->uuconf_u.uuconf_smodem.zdevice);
```

Here, `uuconf_ttype` is the tag field, and the union field associated with 5 is `uuconf_stli`. However, the code erroneously accesses `uuconf_smodem` in the `fprintf` statement. The function contains multiple `fprintf` statements, suggesting that this bug was likely introduced through copy-pasting. After correcting the code to access `uuconf_stli`, Urcrat successfully identifies the tag field.

5.4 RQ2: Correctness

We evaluate the correctness of our approach by checking whether the transformed program is compilable and exhibits the same behavior as the original. For the experiments, we used the fixed code for *glpk-5.0* and *uucp-1.07*, allowing the identification of additional tag fields. Consequently, we examined 30 programs: 29 identified without code fixes and 1, which is *uucp-1.07*, identified only after the code fix. All 30 programs are compilable after transformation. Among 23 programs with test suites, 17 pass the tests post-transformation. Those failed are *gawk-5.2.2*, *grep-3.11*, *make-4.4.1*, *minilisp*, *twemproxy*, and *wget-1.21.4*.

We manually investigated the reasons for the failures and found that only *gawk-5.2.2* and *twemproxy*’s failures result from imprecise identification of tag values in the static analysis. The other failures are due to two specific C code patterns requiring minor manual code fixes after transformation: (1) reading a union field other than the last-written one, and (2) code relying on memory layout. We now discuss the failure reasons for each program.

gawk-5.2.2. The failure arises from tag values not being identified due to intraprocedural analysis.

```
INSTRUCTION *bcalloc(int op, ...) {
  INSTRUCTION *cp = ...; cp->opcode = op; ... return cp;}
```

In this code, `opcode` is the tag field, assigned the argument value. The analysis cannot identify values at the call-site of `bcalloc` as possible `opcode` values. This causes the transformed program to panic when an unknown tag value is passed to `set_opcode`.

twemproxy. The failure is due to the lack of C library modeling, preventing identification of tag values.

```
getaddrinfo(n, s, &h, &ai); si->family = ai->ai_family;
```

Here, `family` is the tag field, and `ai_family` determines its value, set by the `libc` function `getaddrinfo`. While 2 and 10 are possible values, the analysis fails to recognize them as tag values. This issue can be resolved by incorporating library modeling into the analysis.

grep-3.11. It fails because of reading a field not lastly written.

```
fetch_token(token, input, syntax);
c = token->opr.c; if (token->type == 2) return;
```

In this code, `type` is the tag field, and `opr` is the union value. The code first reads `c` and then checks `type`, causing a panic when `get_c` is called in the transformed code. Swapping the order of these statements allows the transformed program to pass the tests.

make-4.4.1. This is also due to reading a field not lastly written.

```
struct function_table_entry { ..., int alloc_fn;
union { fptr1 func_ptr; fptr2 alloc_func_ptr; } fptr; };
if (!entry_p->fptr.func_ptr) abort();
```

The union has two fields, `func_ptr` and `alloc_func_ptr`, with `alloc_fn` as the tag field. The code checks if `func_ptr` is null regardless of `alloc_fn`'s value, exploiting that different function pointer types have the same size and null representation. After transformation, this causes a panic if `alloc_func_ptr` is the last-written field. The code can be fixed as follows to pass the tests post-transformation:

```
if (entry_p->alloc_fn == 0 && !entry_p->fptr.func_ptr ||
    entry_p->alloc_fn==1 && !entry_p->fptr.alloc_func_ptr)
```

minilisp. This failure is caused by different memory layouts of unions and tagged unions.

```
struct Obj { int type; int size; union { ... }; };
Obj *alloc(size_t size, ...) { size += 8; ... }
```

The `alloc` function allocates an `Obj` in a global byte array. It determines the memory size by taking the size of a union field and adding 8 to account for the offset of the union within `Obj`. After transformation, the code becomes as follows:

```
struct Obj {size: i32, c2rust_unnamed: C2RustUnnamed_1}
```

Now, each variant value is at offset 16 due to alignment requirements, resulting in `alloc` allocating insufficient memory. We corrected the code to allocate larger memory, enabling the transformed program to pass the tests.

wget-1.21.4. It also stems from different memory layouts.

```
addr->family = 2; memcpy(&addr->data, tmp, 4);
```

Here, `family` is the tag field, and 2 is associated with the union field `d4` of `addr->data`. The C program writes to `addr->data` because `addr->data` and `addr->data.d4` denote the same address. However, after transformation, `memcpy` overwrites the tag at offset 0 of `addr->data`. We could pass the tests by fixing the code as follows, facilitating `deref_d4_mut` to be called in the transformed code:

```
addr->family = 2; memcpy(&addr->data.d4, tmp, 4);
```

Currently, for end users of the tool, detecting and fixing incorrect translations is challenging. To identify incorrect translations, they must manually inspect the code or run tests. If test suites do not exist, creating new test cases can be costly. Additionally, test cases may not always reveal incorrect translations. Fixing incorrect code is even more challenging, as users need to identify the root cause. The difficulty of this process depends on the root cause. When tag values are not correctly identified or fields not lastly written are read, investigating the cause is relatively straightforward. The test fails due to a panic, allowing users to check which tag value or field read triggered the panic. However, issues related to different memory layouts are more difficult to diagnose since they do not trigger panics. In such cases, users must rely on their debugging skills. We believe it would be beneficial to identify common incorrect translation patterns and design analyses to detect these patterns, thereby providing users with warnings. We leave this as future work.

5.5 RQ3: Efficiency

We evaluate the efficiency of the proposed approach by measuring the execution time of Urcrat, which includes both analysis and transformation, for each program. Fig. 2 presents the execution time

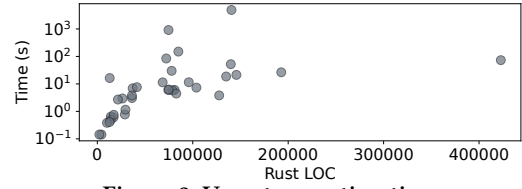


Figure 2: Urcrat execution time

relative to the Rust LOC, with the y-axis displayed in a log scale due to the wide range of execution times. Urcrat efficiently handles most programs, with 31 programs taking less than a minute. The longest execution time is 4,910 seconds for `gawk-5.2.2`. Execution time shows only a weak correlation with code size, as other factors, primarily the complexity of pointer graphs and the number of analyzed functions, also significantly influence execution time.

We also investigate the effectiveness of our selective function analysis in terms of efficiency. For each program, the transformation consumes less than 1% of the total time, as it involves only tree-walking, while the analysis occupies the remaining time. This highlights the importance of reducing analysis time. We calculated the percentage of analyzed functions relative to the total functions in each program, finding a geometric mean of 4.80%. In 28 programs, the analyzed functions constitute less than 10% of the total functions. The highest percentage is 58.23% in the case of `minilisp`. These results indicate that our approach is effective for efficiency.

5.6 RQ4: Code Characteristics

To understand the characteristics of the transformed code, we first evaluate the extent of code changes introduced. We measured the changes in the 30 programs where tag fields are identified. On average, the transformation added 861.8 lines per program due to helper method definitions. Excluding these, an average of 252.4 lines were inserted and 301.8 lines were deleted per program. These results demonstrate the practical utility of our approach, as manually implementing such significant code modifications would be both time-consuming and error-prone.

We also evaluate the applicability of the idiomatic transformation, which avoids inserting method calls. With only the naïve transformation, each program has 194.6 method calls on average. In contrast, applying the idiomatic transformation alongside the naïve transformation reduces this to 124.8 method calls per program, achieving a 36% decrease. Specifically, 38.3 calls are removed by replacing `match` on tag values with `match` on tagged unions, 15.8 calls by replacing `if` with `if-let`, and 15.6 calls by consolidating separate assignments into tagged union construction. The results indicate that the idiomatic transformation improves code quality.

5.7 RQ5: Impact on Performance

We evaluate the impact of replacing unions with tagged unions on the performance of the translated programs. We compare the performance of each Rust program before and after the transformation, measured in terms of the execution time of the test suite. To ensure reliable results, we excluded test suites with execution times shorter than 0.1 seconds, resulting in 20 programs, and computed the average execution time from fifty runs for each program.

The experimental results show that the performance overhead is negligible. On average, the transformed programs were only 0.01% slower than the original ones. Among the 20 programs, 5 ran slightly slower after transformation, while the others were faster. We also performed one-sided t-test with the null hypothesis that the program after transformation is slower than the original program by at least 1%. With a significance level of 0.05, we could reject the null hypothesis for 9 out of 20 programs, implying that the transformation is unlikely to incur observable performance overhead for them. For the remaining 11 programs, we could not reject the null hypothesis, but this does not necessarily mean they have an overhead greater than 1%. Repeating the experiments may provide stronger statistical significance, rejecting the null hypothesis.

5.8 Threats to Validity

Threats to external validity primarily stem from the choice of benchmarks. GNU packages often share common code patterns, which may introduce bias. Although we included projects from GitHub to enhance diversity, this may not fully represent the entire C ecosystem. Notably, large codebases might exhibit different characteristics. Further experiments with a broader range of C programs would provide greater confidence in the generalizability of our approach.

The type errors produced by C2Rust also pose a threat to external validity. Before using our tool, all type errors in C2Rust-generated code must be resolved. Larger programs tend to exhibit more type errors after C2Rust’s translation, requiring significant effort to correct. This could discourage users from adopting both C2Rust and Urcrat, potentially impeding the widespread adoption of our approach. To mitigate this issue, we can improve C2Rust or develop techniques for fixing type errors in C2Rust-generated code.

Threats to construct validity arise from using test suites for correctness and performance evaluation. Passing all tests does not guarantee correctness. However, tests are the most widely used method for practical semantics validation and successfully discovered incorrect behavior in some programs during our evaluation. For performance assessment, the used test suites may be inadequate as they were not designed for performance measurement.

6 RELATED WORK

Improving C2Rust-Generated Code. Several studies have proposed techniques to improve C2Rust’s translation by replacing unsafe features with safe counterparts in Rust. However, none of these studies address the translation of unions, which is the focus of this work. LAERTES [7, 8] and CROWN [48] aim to replace raw pointers with references. LAERTES uses compiler feedback to determine which pointers to replace and the appropriate lifetimes for the references. In contrast, CROWN employs ownership analysis to facilitate the replacement of a larger number of pointers compared to LAERTES. Concrat [14] targets the replacement of some external function calls with equivalent functions from the Rust standard library. It employs dataflow analysis to identify the use of locks, allowing for the substitution of the C lock API with the Rust lock API.

C-to-Rust Translation with Language Models. Recent studies have developed language models capable of translating code [5, 10, 13, 20, 21, 29, 30, 42, 44, 45] or proposed techniques to utilize pre-trained

language models for code translation [25, 47]. Most focus on translating between languages like C++, Java, and Python, rather than C to Rust. A notable exception is Rozière et al. [30], which translates C++ to Rust, but their evaluation uses only small competitive programming solutions that do not involve unions, with only 21% translated correctly. Similarly, Pan et al. [25] use GPT-4 for C-to-Rust translation, applying it to CodeNet [27], which consists of programming problem solutions, achieving 61% correctness. These results indicate that language models frequently generate code that is uncompileable or semantically incorrect, while our approach consistently produces compilable and mostly correct code.

Must-Points-To Analysis. Researchers have studied must-points-to analysis, but their techniques differ from ours in purpose, method, and target language. Our analysis aims to precisely compute struct field values. In contrast, most studies use must-points-to analysis to enhance the precision of other analyses. Altucher and Landi [1] use must-points-to relations to compute def-use relations. Ma et al. [22] and Fink et al. [11] improve may-points-to relations using must-points-to relations, enabling more precise detection of null pointer dereferences and Java typestate checking, respectively. Nikolić and Spoto [24] use must-points-to analysis to improve other analyses, such as nullness and termination analyses. While our approach uses may-points-to analysis as a pre-analysis, some techniques compute may- and must-points-to relations simultaneously. Emami et al. [6] conduct interprocedural may- and must-points-to analysis for compiler optimizations and parallelizations. Sagiv et al. [39] propose a parametric framework that generates a family of shape analyses based on three-valued logic, expressing must-, must-not-, and may-points-to relations. Unlike most studies, including ours, which focus on imperative languages, Jagannathan et al. [17] propose must-points-to analysis for functional languages, with results used for optimizations such as closure conversion. Techniques applicable to general must-points-to analysis have also been explored. Balatsouras et al. [3] present a declarative model in Datalog that expresses a wide range of must-points-to analyses. Kastrinis et al. [19] propose an efficient data structure for pointer graphs.

7 CONCLUSION

In this work, we address the challenge of transforming unions with tags into tagged unions in C2Rust-generated code. This necessitates identifying tag fields for unions and the tag values associated with union fields. To achieve this, we propose a static analysis method that includes must-points-to analysis for computing struct field values and a heuristic for interpreting this information. In addition, we present a code transformation technique that generates idiomatic code for specific patterns while ensuring semantics preservation for the remaining code. Our evaluation shows that the proposed approach is precise, mostly correct, and scalable.

ACKNOWLEDGMENTS

This research was supported by the National Research Foundation of Korea (NRF) (2022R1A2C200366011 and 2021R1A5A1021944), the Institute for Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (2022-0-00460, 2023-2020-0-01819, and 2024-00337703), and Samsung Electronics Co., Ltd (G01210570).

REFERENCES

- [1] Rita Z. Altucher and William Landi. 1995. An extended form of must alias analysis for dynamic allocation. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL '95). Association for Computing Machinery, New York, NY, USA, 74–84. <https://doi.org/10.1145/199448.199466>
- [2] Lars Ole Andersen. 1994. Program Analysis and Specialization for the C Programming Language. *PhD Thesis, University of Copenhagen* (1994).
- [3] George Balatsouras, Kostas Ferles, George Kastrinis, and Yannis Smaragdakis. 2017. A Datalog model of must-alias analysis. In *Proceedings of the 6th ACM SIGPLAN International Workshop on State Of the Art in Program Analysis* (Barcelona, Spain) (SOAP 2017). Association for Computing Machinery, New York, NY, USA, 7–12. <https://doi.org/10.1145/3088515.3088517>
- [4] Haogang Chen, Yandong Mao, Xi Wang, Dong Zhou, Nickolai Zeldovich, and M. Frans Kaashoek. 2011. Linux kernel vulnerabilities: state-of-the-art defenses and open problems. In *Proceedings of the Second Asia-Pacific Workshop on Systems* (Shanghai, China) (APSys '11). Association for Computing Machinery, New York, NY, USA, Article 5, 5 pages. <https://doi.org/10.1145/2103799.2103805>
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebbgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374* [cs.LG] <https://arxiv.org/abs/2107.03374>
- [6] Maryam Emami, Rakesh Ghiya, and Laurie J. Hendren. 1994. Context-sensitive interprocedural points-to analysis in the presence of function pointers. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation* (Orlando, Florida, USA) (PLDI '94). Association for Computing Machinery, New York, NY, USA, 242–256. <https://doi.org/10.1145/178243.178264>
- [7] Mehmet Emre, Peter Boyland, Aesha Parekh, Ryan Schroeder, Kyle Dewey, and Ben Hardekopf. 2023. Aliasing Limits on Translating C to Safe Rust. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 94 (apr 2023), 29 pages. <https://doi.org/10.1145/3586046>
- [8] Mehmet Emre, Ryan Schroeder, Kyle Dewey, and Ben Hardekopf. 2021. Translating C to Safer Rust. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 121 (oct 2021), 29 pages. <https://doi.org/10.1145/3485498>
- [9] Mazdak Farrokhzad. 2018. Tracking issue for rERC 2497, “if- and while-let-chains, take 2”. <https://github.com/rust-lang/rust/issues/53667>
- [10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [11] Stephen J. Fink, Eran Yahav, Nurit Dor, G. Ramalingam, and Emmanuel Geay. 2008. Effective typestate verification in the presence of aliasing. *ACM Trans. Softw. Eng. Methodol.* 17, 2, Article 9 (may 2008), 34 pages. <https://doi.org/10.1145/1348250.1348255>
- [12] GNU. 2024. GNU Package Blurbs. <https://www.gnu.org/manual/blurbs.html>
- [13] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021*. OpenReview.net. <https://openreview.net/forum?id=jLoC4ez43PZ>
- [14] Jaemin Hong and Sukyoung Ryu. 2023. Concrat: An Automatic C-to-Rust Lock API Translator for Concurrent Programs. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, Melbourne, Victoria, Australia, 716–728. <https://doi.org/10.1109/ICSE48619.2023.00069>
- [15] Jaemin Hong and Sukyoung Ryu. 2024. To Tag, or Not to Tag: Translating C's Unions to Rust's Tagged Unions (Artifact). <https://doi.org/10.5281/zenodo.13373683>
- [16] Tim Hutt. 2021. Would Rust secure cURL? <https://blog.timhutt.co.uk/curl-vulnerabilities-rust/>
- [17] Suresh Jagannathan, Peter Thiemann, Stephen Weeks, and Andrew Wright. 1998. Single and loving it: must-alias analysis for higher-order languages. In *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '98). Association for Computing Machinery, New York, NY, USA, 329–341. <https://doi.org/10.1145/268946.268973>
- [18] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2017. RustBelt: Securing the Foundations of the Rust Programming Language. *Proc. ACM Program. Lang.* 2, POPL, Article 66 (dec 2017), 34 pages. <https://doi.org/10.1145/3158154>
- [19] George Kastrinis, George Balatsouras, Kostas Ferles, Nefeli Prokopiaki-Kostopoulou, and Yannis Smaragdakis. 2018. An efficient data structure for must-alias analysis. In *Proceedings of the 27th International Conference on Compiler Construction* (Vienna, Austria) (CC 2018). Association for Computing Machinery, New York, NY, USA, 48–58. <https://doi.org/10.1145/3178372.3179519>
- [20] Marie-Anne Lachaux, Baptiste Roziere, Marc Szafraniec, and Guillaume Lample. 2024. DOBF: a deobfuscation pre-training objective for programming languages. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA, Article 1147, 13 pages.
- [21] Fang Liu, Jia Li, and Li Zhang. 2023. Syntax and Domain Aware Model for Unsupervised Program Translation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 755–767. <https://doi.org/10.1109/ICSE48619.2023.00072>
- [22] Xiaodong Ma, Ji Wang, and Wei Dong. 2008. Computing Must and May Alias to Detect Null Pointer Dereference. In *Leveraging Applications of Formal Methods, Verification and Validation*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 252–261.
- [23] Nicholas D. Matsakis and Felix S. Klock. 2014. The Rust Language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology* (Portland, Oregon, USA) (HILT '14). Association for Computing Machinery, New York, NY, USA, 103–104. <https://doi.org/10.1145/2663171.2663188>
- [24] Durica Nikolić and Fausto Spoto. 2012. Definite Expression Aliasing Analysis for Java Bytecode. In *Theoretical Aspects of Computing – ICTAC 2012*, Abhik Roychoudhury and Meenakshi D'Souza (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 74–89.
- [25] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 82, 13 pages. <https://doi.org/10.1145/3597503.3639226>
- [26] David J. Pearce, Paul H.J. Kelly, and Chris Hankin. 2007. Efficient field-sensitive pointer analysis of C. *ACM Trans. Program. Lang. Syst.* 30, 1 (nov 2007), 4–es. <https://doi.org/10.1145/1290520.1290524>
- [27] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir R. Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. 2021. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/a5bfc9e07964f8dddeb95fc584cd965d-Abstract-round2.html>
- [28] H. Gordon Rice. 1953. Classes of recursively enumerable sets and their decision problems. *Trans. Amer. Math. Soc.* 74 (1953), 358–366. <https://doi.org/10.1090/s0002-9947-1953-0053041-6>
- [29] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 1730, 11 pages.
- [30] Baptiste Roziere, Jie Zhang, François Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. 2022. Leveraging Automated Unit Tests for Unsupervised Code Translation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022*. OpenReview.net. <https://openreview.net/forum?id=cmt-6KtR4c4>
- [31] Rust. 2024. Guide to Rustc Development. <https://rustc-dev-guide.rust-lang.org/>
- [32] Rust. 2024. Guide to Rustc Development: The HIR. <https://rustc-dev-guide.rust-lang.org/hir.html>
- [33] Rust. 2024. Guide to Rustc Development: The MIR. <https://rustc-dev-guide.rust-lang.org/mir/index.html>
- [34] Rust. 2024. Rust by Example: if let. https://doc.rust-lang.org/rust-by-example/flow_control/if_let.html
- [35] Rust. 2024. The Rust Programming Language: Defining an Enum. <https://doc.rust-lang.org/book/ch06-01-defining-an-enum.html>
- [36] Rust. 2024. The Rust Programming Language: Pattern Syntax. <https://doc.rust-lang.org/book/ch18-03-pattern-syntax.html>
- [37] Rust. 2024. The Rust Programming Language: Unsafe Rust. <https://doc.rust-lang.org/book/ch19-01-unsafe-rust.html>
- [38] Rust. 2024. The Rust Reference: Unions. <https://doc.rust-lang.org/reference/items/unions.html>

- [39] Mooly Sagiv, Thomas Reps, and Reinhard Wilhelm. 1999. Parametric shape analysis via 3-valued logic. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Antonio, Texas, USA) (POPL '99). Association for Computing Machinery, New York, NY, USA, 105–118. <https://doi.org/10.1145/292540.292552>
- [40] Bjarne Steensgaard. 1996. Points-to Analysis by Type Inference of Programs with Structures and Unions. In *Proceedings of the 6th International Conference on Compiler Construction (CC '96)*. Springer-Verlag, Berlin, Heidelberg, 136–150.
- [41] Bjarne Steensgaard. 1996. Points-to analysis in almost linear time. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (St. Petersburg Beach, Florida, USA) (POPL '96). Association for Computing Machinery, New York, NY, USA, 32–41. <https://doi.org/10.1145/237721.237727>
- [42] Marc Szafraniec, Baptiste Rozière, Hugh Leather, Patrick Labatut, François Charton, and Gabriel Synnaeve. 2023. Code Translation with Compiler Representations. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023*. OpenReview.net. <https://openreview.net/forum?id=XomEU3eNeSQ>
- [43] Gavin Thomas. 2019. A proactive approach to more secure code. <https://msrc-blog.microsoft.com/2019/07/16/a-proactive-approach-to-more-secure-code>.
- [44] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] <https://arxiv.org/abs/2302.13971>
- [45] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 8696–8708. <https://doi.org/10.18653/v1/2021.emnlp-main.685>
- [46] Frances Wingerter. 2022. C2Rust is Back. <https://immunant.com/blog/2022/06/back/>.
- [47] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *Proc. ACM Softw. Eng.* 1, FSE, Article 71 (jul 2024), 24 pages. <https://doi.org/10.1145/3660778>
- [48] Hanliang Zhang, Cristina David, Yijun Yu, and Meng Wang. 2023. Ownership Guided C to Rust Translation. In *Computer Aided Verification*, Constantin Enea and Akash Lal (Eds.). Springer Nature Switzerland, Cham, 459–482.